

Lecture 7: Adversarial and Diffusion Models
GANs, Wasserstein Distance, Diffusion
Generating Spin Configurations

Deep Learning for Scientific Discovery

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków



Recap: Variational Autoencoders

- a generative model learns $p(\mathbf{x})$ from samples and draws new ones
- VAE: encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$, decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$, prior $\mathcal{N}(0, I)$; trained on reconstruction + KL; sampling: $\mathbf{z} \sim \mathcal{N}(0, I)$, decode
- conditional VAE: choose $\mathbf{c}$ (what), draw $\mathbf{z}$ (how); β-VAE: one latent coordinate per feature
- VAE samples are somewhat blurry, and the likelihood is only bounded

Today: generators trained without a likelihood (GANs), by removing noise (diffusion), and checked against physics.

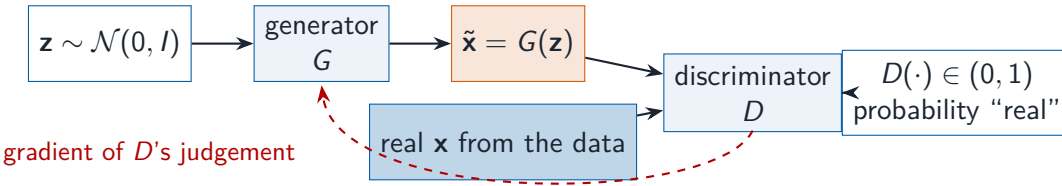
1

Generative Adversarial Networks



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Generative Adversarial Network



- **generator** G: turns random noise $\mathbf{z}$ into a sample $\tilde{\mathbf{x}} = G(\mathbf{z})$; it never sees the data
- **discriminator** D: a binary classifier, $D(\mathbf{x})$ = probability that $\mathbf{x}$ comes from the data and not from G
- trained against each other: G learns from D's gradient to produce samples that D takes for real
- no formula for $p_G(\mathbf{x})$: the GAN learns to *sample*, not to evaluate a probability

I. J. Goodfellow et al., *Generative Adversarial Networks*, NeurIPS 2014, arXiv:1406.2661

The GAN Loss

D is an ordinary classifier (real $= 1$, generated $= 0$); G is trained to make it wrong. One value, maximised by D and minimised by G:

$$V(D,G) = \underbrace{\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\big[\log D(\mathbf{x})\big]}_{D \text{ recognises real data}} + \underbrace{\mathbb{E}_{\mathbf{z}\sim\mathcal{N}(0,I)}\big[\log\big(1 - D(G(\mathbf{z}))\big)\big]}_{D \text{ recognises generated data}}, \qquad \min_G \max_D V(D,G)$$

$-V$: the binary cross-entropy of D; G now denotes the generator. Training alternates gradient steps of D and of G.

- in practice G minimises $-\log D(G(\mathbf{z}))$, not $\log(1 - D(G(\mathbf{z})))$: same optimum, strong gradient when D rejects the samples easily, early in training (used in our notebook)
- with the best D, $V = -\log 4 + 2\,\mathrm{JS}(p_{\text{data}} \,\|\, p_G)$, Jensen–Shannon divergence $\mathrm{JS}(p \,\|\, q) = \frac{1}{2}\mathrm{KL}(p \,\|\, m) + \frac{1}{2}\mathrm{KL}(q \,\|\, m)$, $m = \frac{1}{2}(p+q)$: 0 only at $p_G = p_{\text{data}}$, where the game ends

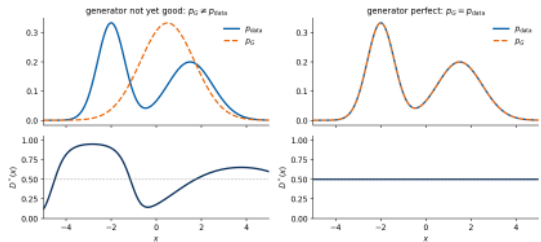
I. J. Goodfellow et al., *Generative Adversarial Networks*, NeurIPS 2014, arXiv:1406.2661

The Best Discriminator

For a fixed generator with density p_G, the D that maximises V is

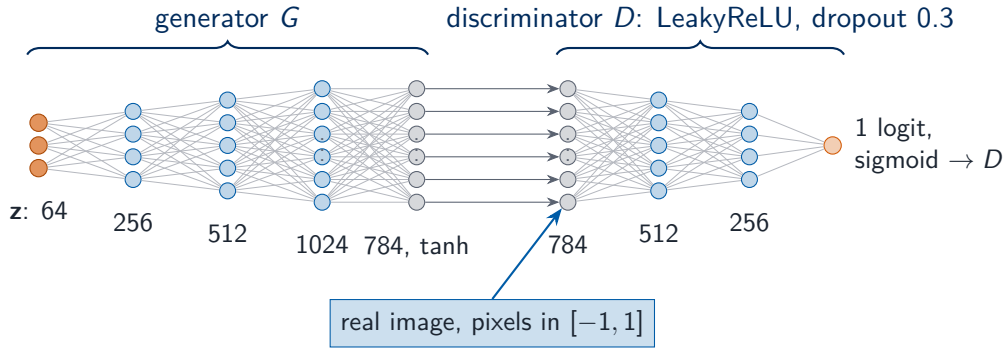
$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_G(\mathbf{x})}$$

the probability that a point at $\mathbf{x}$ is real: at each $\mathbf{x}$, D maximises $a \log y + b \log(1-y)$, $a = p_{\text{data}}(\mathbf{x})$, $b = p_G(\mathbf{x})$, with maximum at $y = a/(a+b)$. $D^* > \frac{1}{2}$: G puts too little probability there; $D^* < \frac{1}{2}$: too much; $p_G = p_{\text{data}}$: $D^* = \frac{1}{2}$ everywhere.



I. J. Goodfellow et al., *Generative Adversarial Networks*, NeurIPS 2014, arXiv:1406.2661 (Proposition 1)

GAN on MNIST: the Networks

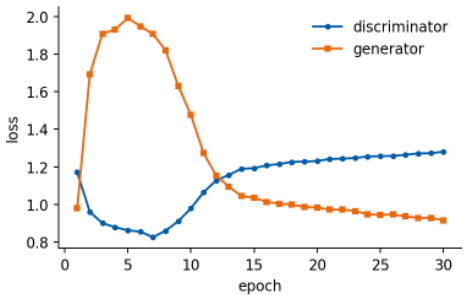


One training step, on a mini-batch of real images and of $G(\mathbf{z})$:

1. update D: binary cross-entropy with labels real $= 1$, generated $= 0$
2. update G, with D frozen: label its samples "real" $(= 1)$ and backpropagate through D into G (Adam, learning rate $2 \cdot 10^{-4}$, 30 epochs)

A GAN on MNIST



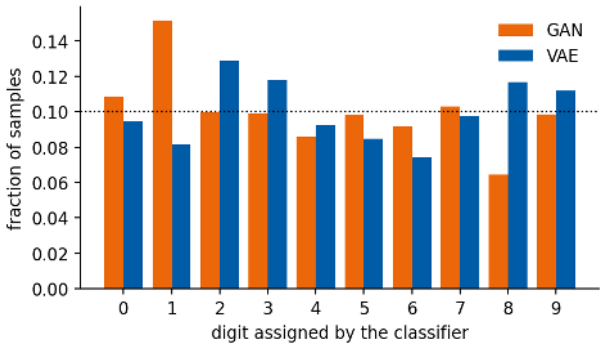


- top: the same 16 random $\mathbf{z}$ after 1, 5 and 30 epochs; the samples sharpen
- the loss curves do not show it: they measure a game between two networks, not the quality of the samples

Difficulties of GAN Training

- **a game, not a minimisation:** the solution is a saddle point (a maximum for D and a minimum for G at the same time); no loss decreases steadily, the two networks can oscillate, and the loss curves say little about the quality of the samples
- **vanishing gradients:** when D separates real from generated perfectly, $\log(1 - D(G(\mathbf{z})))$ saturates and G gets almost no gradient
- **mode collapse:** G finds a few samples that fool D and produces only those, e.g. only some of the digits
- **sensitivity:** learning rates, activations and the balance between G and D matter; our first version (ReLU, sigmoid output, no dropout) collapsed onto a single blob within one epoch

Mode Collapse



- the MNIST classifier labels 2000 new samples of the GAN and 2000 of the VAE
- a generator that covers the data produces each digit about as often as in MNIST, roughly 10% (dotted line; from 9% for 5s to 11% for 1s)
- this GAN produces every digit, between 6% and 15% of the samples: no mode collapse

The Wasserstein Distance

On a line: the area between the cumulative distribution functions F,

$$W(p, q) = \int |F_p(x) - F_q(x)|\, dx,$$

the amount of probability moved times the distance it travels (earth mover's distance).

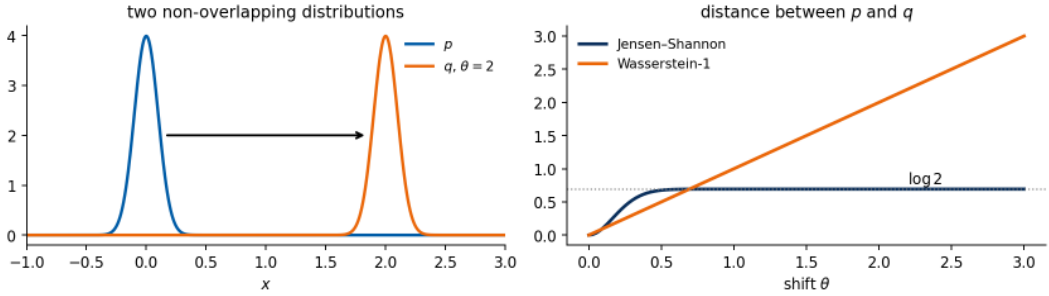
In general (Kantorovich–Rubinstein):

$$W(p, q) = \max_{\|f\|_L \le 1} \mathbb{E}_p[f] - \mathbb{E}_q[f]$$

over 1-Lipschitz f: $|f(x) - f(y)| \le |x - y|$. In a WGAN the **critic** f maximises $\mathbb{E}_{\text{data}}[f] - \mathbb{E}_{\mathbf{z}}[f(G(\mathbf{z}))]$ (a score, not a probability; the slope limit is kept by a gradient penalty), and G minimises $-\mathbb{E}_{\mathbf{z}}[f(G(\mathbf{z}))]$.

M. Arjovsky, S. Chintala, L. Bottou, *Wasserstein GAN*, ICML 2017, arXiv:1701.07875; I. Gulrajani et al., *Improved Training of Wasserstein GANs*, NeurIPS 2017, arXiv:1704.00028

Wasserstein and Jensen–Shannon Divergence

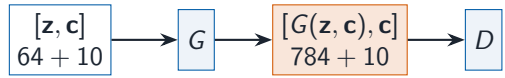


Two bumps a distance θ apart. The Jensen–Shannon divergence (what the original GAN loss measures at the best D) stays at $\log 2$ once they do not overlap: no gradient. $W = \theta$: it says how far, at every distance.

M. Arjovsky, S. Chintala, L. Bottou, *Wasserstein GAN*, ICML 2017, arXiv:1701.07875; I. Gulrajani et al., *Improved Training of Wasserstein GANs*, NeurIPS 2017, arXiv:1704.00028

Conditional GAN





- **c** (one-hot digit) is appended to the input of *both* networks; $D(\mathbf{x}, \mathbf{c})$ judges the *pair*: is this a real image of digit **c**?
- a realistic 3 sent with $\mathbf{c}$ = "7" is rejected, so G must respect the condition
- rows: requested digits 0 to 9; a classifier of real digits recognises 0.96 of 200 samples per digit as requested (conditional VAE, same test: 0.96)

M. Mirza, S. Osindero, *Conditional Generative Adversarial Nets*, arXiv:1411.1784

VAE and GAN

	VAE	GAN
objective	maximise the ELBO $\leq \log p_\theta(\mathbf{x})$	$\min_G \max_D V(D, G)$
training	one loss that decreases	a game between two networks
latent space	an encoder maps data to it	no encoder
likelihood	a lower bound on $\log p_\theta(\mathbf{x})$	none
samples	often blurry	sharper edges, but pixel noise and artefacts
failure	blurry averages	unstable training, mode collapse

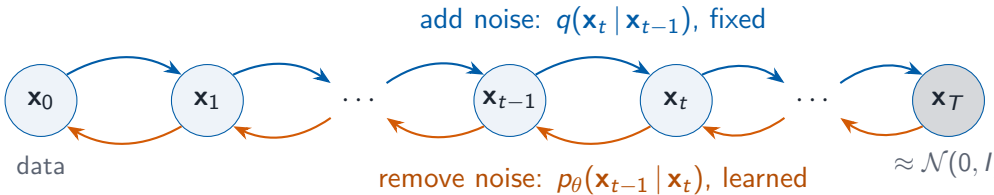
2

Diffusion Models



JAGIELLONIAN UNIVERSITY
IN KRAKÓW

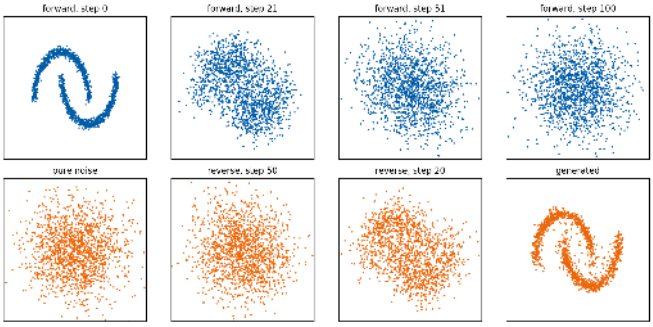
Diffusion Models



Forward: a little Gaussian noise per step until only noise is left (no learning). Reverse: a network removes the noise one step at a time; started from fresh noise, it produces a new sample.

J. Sohl-Dickstein et al., *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*, ICML 2015, arXiv:1503.03585; J. Ho, A. Jain, P. Abbeel, *Denoising Diffusion Probabilistic Models*, NeurIPS 2020, arXiv:2006.11239

Diffusion on Two Moons



Top row: the forward process destroys the two moons, one small step at a time, until only noise is left. Bottom row: the learned reverse process turns fresh noise into new points on the two moons.

The Forward Process

One step, with a fixed schedule $\beta_1 < \cdots < \beta_T$:

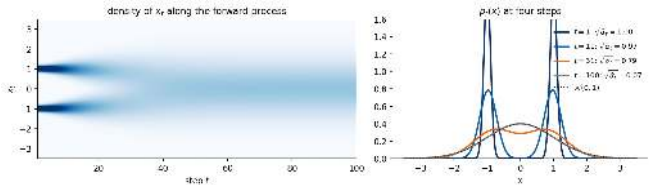
$$\mathbf{x}_t = \sqrt{1-\beta_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I)$$

$(\sqrt{1-\beta_t})^2 + (\sqrt{\beta_t})^2 = 1$: data of variance 1 keep variance 1.

t steps at once (sums of independent Gaussians are Gaussian):

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon, \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$$

$\bar{\alpha}_T \approx 0$: the last step is pure noise.

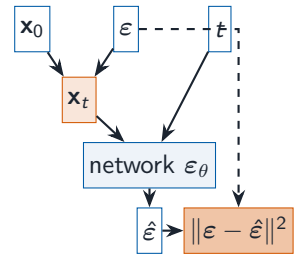


1D data at ± 1, $T = 100$, β_t from 10^{-4} to 0.1: $\sqrt{\bar{\alpha}_T} = 0.075$. The two peaks merge into one $\mathcal{N}(0, 1)$.

Training a Diffusion Model

A data sample $\mathbf{x}_0$, a random step t, a known noise $\varepsilon \sim \mathcal{N}(0, I)$: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon$. The network $\varepsilon_\theta(\mathbf{x}_t, t)$ **predicts the noise**:

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0, t, \varepsilon} \left\| \varepsilon - \varepsilon_\theta(\mathbf{x}_t, t) \right\|^2$$



an ordinary regression: one loss that decreases, stable training.

Generation is slow: many denoising steps. Behind today's image generators, and in science: protein structures (RFdiffusion), crystals (MatterGen), weather ensembles (GenCast).

J. L. Watson et al., *De novo design of protein structure and function with RFdiffusion*, Nature **620**, 1089 (2023), doi:10.1038/s41586-023-06415-8; C. Zeni et al., *A generative model for inorganic materials design*, Nature **639**, 624 (2025), doi:10.1038/s41586-025-08628-5; I. Price et al., *Probabilistic weather forecasting with machine learning*, Nature **637**, 84 (2025), doi:10.1038/s41586-024-08252-9

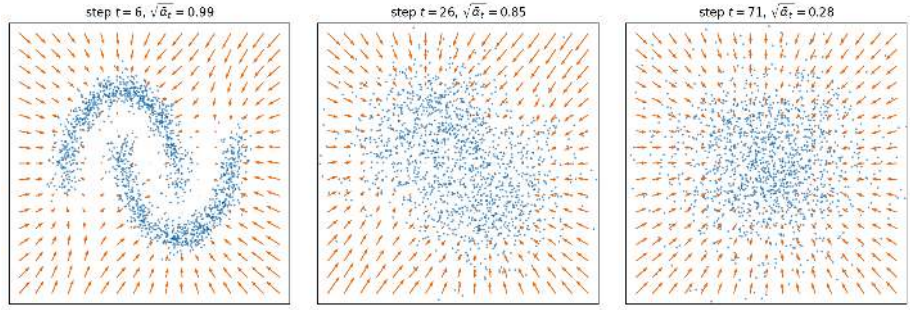
The Reverse Step

From $\mathbf{x}_T \sim \mathcal{N}(0, I)$, for $t = T, \dots, 1$, with $\alpha_t = 1 - \beta_t$ and fresh $\boldsymbol{\xi} \sim \mathcal{N}(0, I)$ (none at $t = 1$):

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\Big(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\varepsilon_\theta(\mathbf{x}_t, t)\Big) + \sqrt{\beta_t}\,\boldsymbol{\xi}$$

From $\hat{\varepsilon}$ the clean sample can be estimated, $\hat{\mathbf{x}}_0 = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\varepsilon})/\sqrt{\bar{\alpha}_t}$, and the step moves $\mathbf{x}_t$ a little towards it, plus fresh noise. The predicted noise is also a direction, with p_t the distribution of $\mathbf{x}_t$: $-\varepsilon_\theta/\sqrt{1-\bar{\alpha}_t} \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, the **score**, which points to higher density of the noisy data.

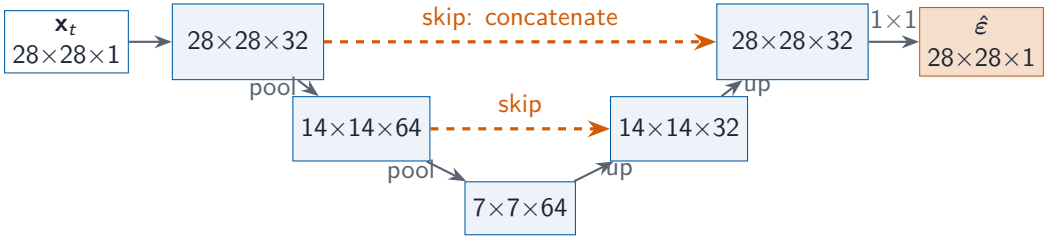
The Learned Score



Blue: noisy two-moons data at three steps; orange: the learned arrows. At every step they point from empty regions towards the noisy data, which change from two moons (small t) to one broad cloud (large t).

Y. Song et al., *Score-Based Generative Modeling through Stochastic Differential Equations*, ICLR 2021, arXiv:2011.13456

The U-Net Noise Predictor

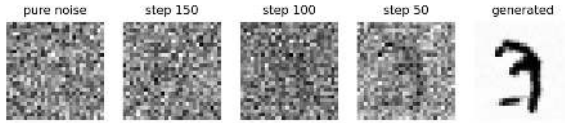


step $t \to 64$ numbers $\sin(\omega_k t), \cos(\omega_k t)$ at 32 frequencies ω_k:
a smooth code of t, added to the features in every block

Each block: two 3×3 convolutions; the final 1×1 convolution mixes the 32 channels of each pixel into one number. The shrinking path sees larger regions; the skip connections carry the fine details to the growing path. The network predicts the noise of every pixel.

O. Ronneberger, P. Fischer, T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI 2015, arXiv:1505.04597

Diffusion on MNIST



one generated digit, from pure noise through the reverse steps



- $T = 200$ steps, β_t from $5 \cdot 10^{-4}$ to 0.1: $\sqrt{\bar{\alpha}_T} = 0.0055$; one image = 200 passes of the U-Net
- trained on the 60 000 training digits in about 26 minutes on a CPU
- all ten digits appear, most readable (mean classifier confidence 0.80)

3

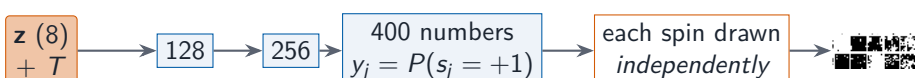
Generating Ising Configurations

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Generating Spin Configurations at a Chosen Temperature

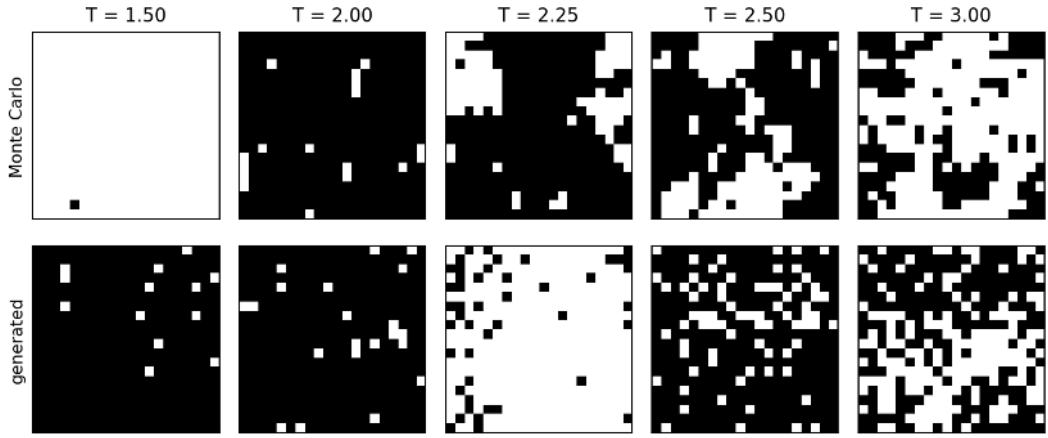
For spins the true distribution is known, $p(\mathbf{s} \mid T) = e^{-H(\mathbf{s})/T}/Z$, $H(\mathbf{s}) = -\sum_{\langle ij \rangle} s_i s_j$ (sum over nearest-neighbour pairs, $J = k_B = 1$; Lecture 4); Metropolis Monte Carlo samples it, exactly in the limit of a long run. Can a network learn it from samples alone?

- **data:** Monte Carlo configurations of a 20×20 lattice, 51 temperatures from 1.0 to 3.5, 200 configurations each
- **model:** conditional VAE, $c = T$, 8 latent numbers; spins ± 1 as pixels $0/1$
- **physical test:** compare $\langle |M| \rangle$ and the energy per spin of the generated configurations with Monte Carlo, at every T



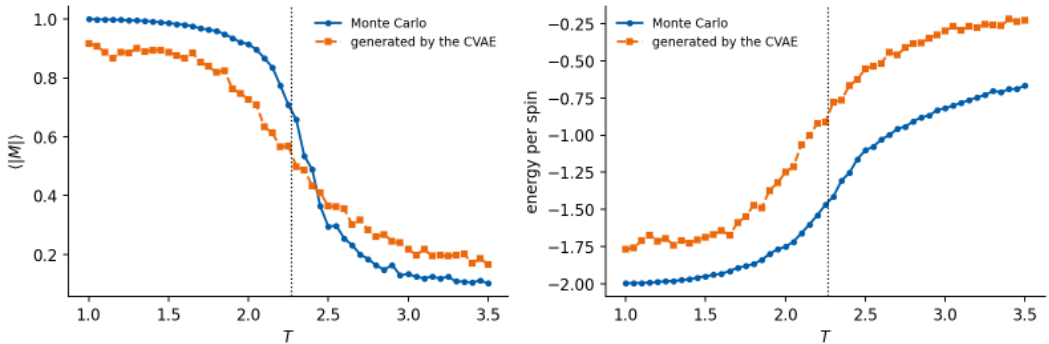
$$p_\theta(\mathbf{s} \mid \mathbf{z}, T) = \prod_{j=1}^{400} y_j^{(1+s_j)/2} (1 - y_j)^{(1-s_j)/2}$$

Generated Spin Configurations



Configurations generated by the conditional VAE at chosen temperatures, next to Monte Carlo configurations at the same temperatures.

Observables of Generated Spin Configurations



$\langle |M| \rangle$ follows the drop around T_c; the energy is too high at every T (largest gap 0.63 per spin, at $T = 2.3$). Given $\mathbf{z}$, the spins are drawn independently, so isolated flipped spins appear where Monte Carlo has compact domains: the nearest-neighbour correlations $\langle s_i s_j \rangle$, which fix the energy, are lost. **A physical test shows what a picture can hide.**

Summary

- **GAN**: a generator against a discriminator; sharper edges than a VAE, hard training; the Wasserstein distance gives a usable signal
- **diffusion**: add noise in many small steps, learn to remove it; the noise predictor is a score, pointing to higher density
- **physical tests**: generated spin configurations checked against Monte Carlo through observables, correlations and classifiers

Tutorial: Hands-on

Hands-on

- train the GAN with the saturating generator loss: does training start?
- use fewer diffusion steps: how do the generated samples change?
- condition the Ising generator on a temperature between two training temperatures: are $\langle |M| \rangle$ and the energy right?

Thank You!

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar



